

Program 8: Working with Gradle: Setting Up a Gradle Project

Setting Up a Gradle Project

Step 1: To create a basic Gradle Project, run following commands:

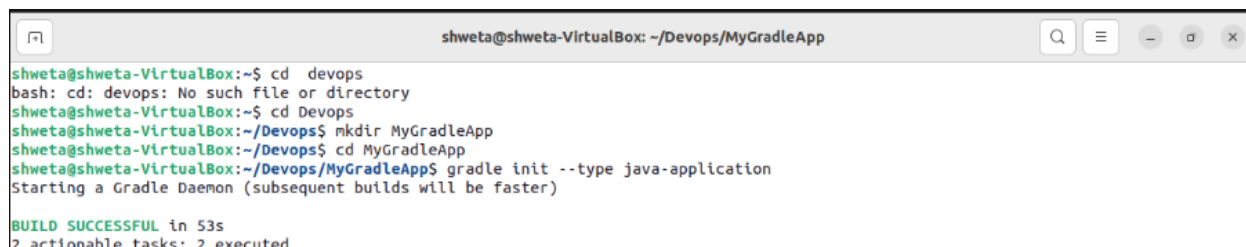
mkdir MyGradleApp: to create a Gradle

project directory **cd MyGradleApp:** to

change directory to project directory **Step**

2: gradle init --type java-application

gradle init --type java-application where **gradle** is the build tool, **init** creates a new project, **--type** specifies the project kind, and **java-application** means a basic Java app project with structure and files is generated.

A terminal window titled 'shweta@shweta-VirtualBox: ~/Devops/MyGradleApp' showing the following commands and output:

```
shweta@shweta-VirtualBox:~$ cd devops
bash: cd: devops: No such file or directory
shweta@shweta-VirtualBox:~$ cd Devops
shweta@shweta-VirtualBox:~/Devops$ mkdir MyGradleApp
shweta@shweta-VirtualBox:~/Devops$ cd MyGradleApp
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ gradle init --type java-application
Starting a Gradle Daemon (subsequent builds will be faster)
BUILD SUCCESSFUL in 53s
2 actionable tasks: 2 executed
```

Step 3: **tree** is a command used to **display the folder structure in a hierarchical (tree-like) format**, showing all directories and files inside a path.

Files Breakdown – instead of pom.xml we have four main files that have several important information regarding the gradle application:

```
2 actionable tasks: 2 executed
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle
└── src
    ├── main
    │   └── java
    │       └── App.java
    └── test
        └── java
            └── AppTest.java

7 directories, 8 files
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$
```

- **build.gradle** – Main build script. Defines project settings, dependencies, and plugins
- **settings.gradle** – sets the project name and can include subprojects (for multi-project builds)
- **gradlew & gradlew.bat** – Gradle Wrapper scripts. Use these to run Gradle without requiring a system install. gradlew for Unix, .bat for Windows.

By default, build.gradle looks like:

```
Open  build.gradle
~/Devops/MyGradleApp

1 /*
2  * This build file was generated by the Gradle 'init' task.
3  *
4  * This generated file contains a sample Java project to get you started.
5  * For more details take a look at the Java Quickstart chapter in the Gradle
6  * user guide available at https://docs.gradle.org/4.4.1/userguide/tutorial\_java\_projects.html
7  */
8
9 // Apply the java plugin to add support for Java
10 apply plugin: 'java'
11
12 // Apply the application plugin to add support for building an application
13 apply plugin: 'application'
14
15 // In this section you declare where to find the dependencies of your project
16 repositories {
17     // Use jcenter for resolving your dependencies.
18     // You can declare any Maven/Ivy/file repository here.
19     jcenter()
20 }
21
22 dependencies {
23     // This dependency is found on compile classpath of this component and consumers.
24     compile 'com.google.guava:guava:23.0'
25
26     // Use JUnit test framework
27     testCompile 'junit:junit:4.12'
28 }
29
30 // Define the main class for the application
31 mainClassName = 'App'
32
```

Step 4: Go to terminal and run following commands:

gradle build: it compiles the source code and package the app

```
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ gedit build.gradle
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ gedit build.gradle
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ gradle build
```

```
BUILD SUCCESSFUL in 4s
7 actionable tasks: 7 up-to-date
```

gradle run: Run the default Hello World application

```
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ gradle run
```

```
Task :run
hello world.
```

Step 5: Gradle allows you to define and customize **tasks**, where **Gradle** is the build tool, **tasks** are individual units of work like compiling, testing, or packaging, **built-in tasks** are already provided by Gradle, and you can also create **custom tasks** by adding code in the **build.gradle** file, which is the main configuration file; using **gedit** (a text editor) you can open this file and add your own task definitions.

doLast is used to add the action to the end of the task's execution sequence so it runs last, whereas

doFirst adds the action at the beginning so it runs first.

Dependency Management and Task Automation

Dependency Management: Gradle manages project libraries (dependencies) by downloading them from repositories like Maven Central or JCenter, ensuring the correct versions are included in the project.

Task Automation: Gradle automates tasks such as compiling code, running tests, generating JAR files, cleaning builds, etc. You can use built-in tasks or create custom tasks. There are some built-in tasks in Gradle:

```
gradle build # Full build
```

```
gradle test    # Runs tests
gradle run     # Runs the application
```

Setting Up Jenkins Pipeline for MyGradleApp

Step 1: To deploy **MyGradleApp**, open the terminal, cd into the **MyGradleApp** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

```
pipeline {
  agent any // Use any available agent

  tools {
    gradle 'Gradle' // Ensure this matches the name configured in Jenkins

    jdk 'JDK'
  }

  stages {

    stage('Checkout') { steps {

      git branch: 'master', url: 'https://github.com/Shweta311204/MyGradleApp.git'
    }
  }

  Stage('Build') { steps {
    sh 'gradle build' // Run Gradle build
  }
}

  stage('Test') { steps {
    sh 'gradle test' // Run unit tests
  }
}

  stage('Run Application') { steps {
```

// Start the JAR application

```

sh 'gradle run'
}
}
}

post {

success {

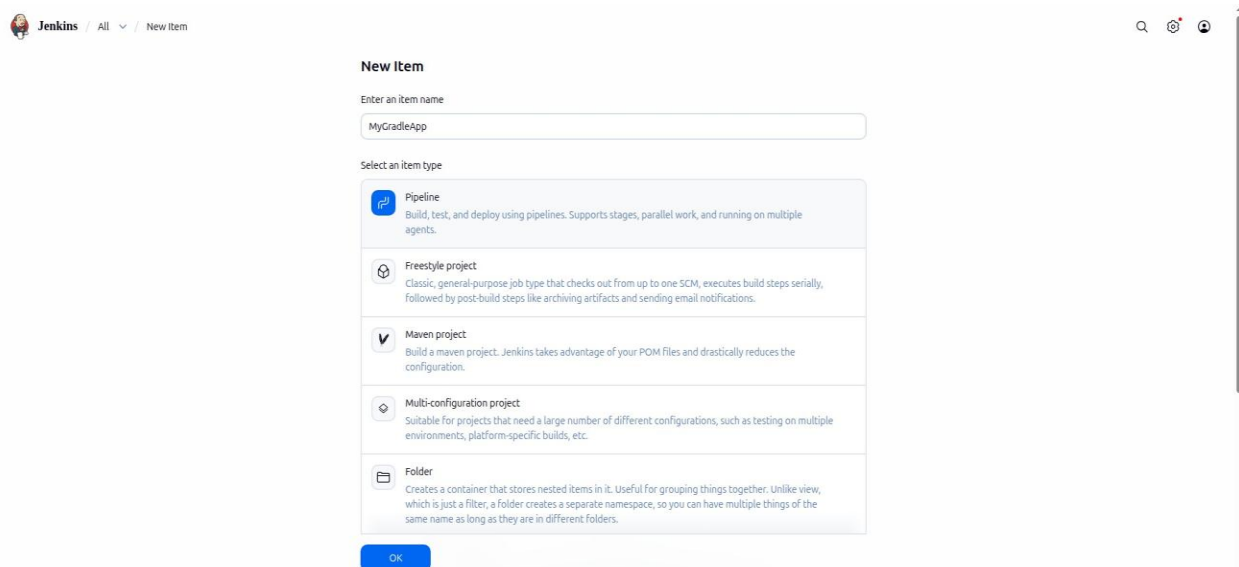
echo 'Build and deployment successful!'
}

failure {

echo 'Build failed!'
}
}
}
}

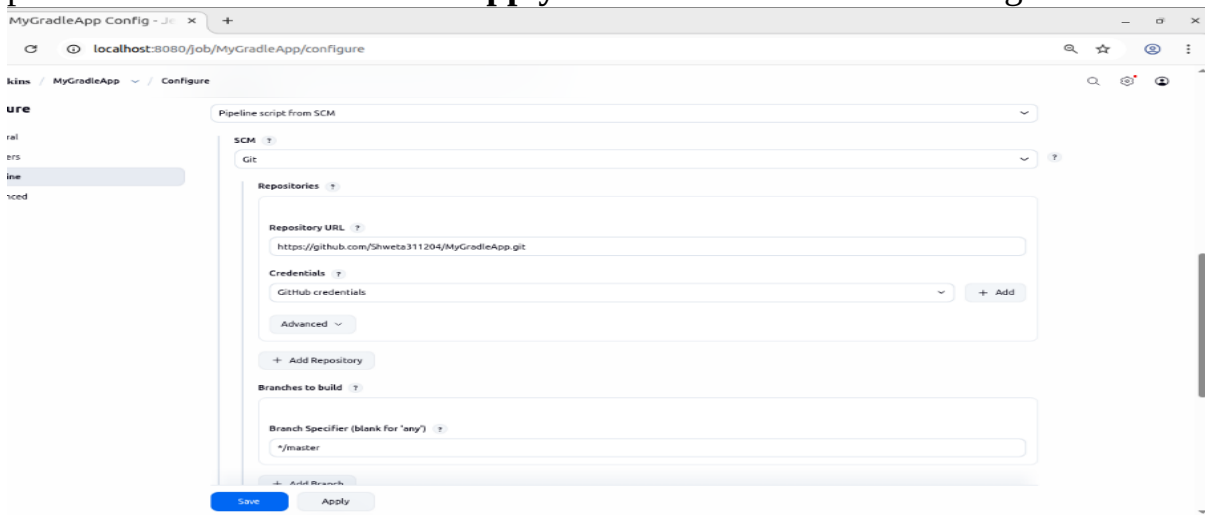
```

Step 2: To deploy **MyGradleApp**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on “**New Item**” to create a job, enter the item name (e.g., *MyGradleApp*), and select **Pipeline** as the project type. This option allows you to



build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page.

Setup 3: In **MyGradleApp**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyGradleApp**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MyGradleApp folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.



Step 4: After clicking **Apply** and **Save**, go back to **MyGradlePipelineApp** in Jenkins and click **Build Now**; the build completes successfully.

